

|             |                                 |
|-------------|---------------------------------|
| Title       | BUS Interface Integration Guide |
| Project     | All                             |
| Description | General한 BUS Interface 사용 방법    |
| Author      | holelee                         |
| Date        | 19/Feb/2007                     |

### (\*) General한 BUS Interface

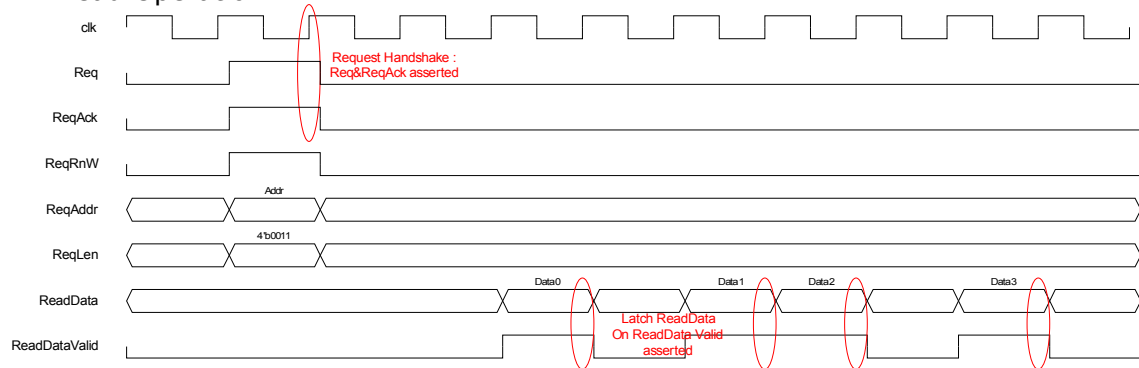
일반적인 AXI Master가 사용할 수 있도록 DMA용 BUS Interface를 설계했다. 이것의 Integration 방법에 대해서 살펴 본다.

### (\*) Features

- Simple Interface
- Configurable Read-Only/Write-Only/Read-Write
  - 세 가지 모드로 Configuration 가능하다.
- Maximum 16 beat Burst
- AXI Compliant
  - Internal 4KB Boundary Check & Split Request
- Independent Request/Data
  - AXI와 마찬가지로 Request와 Data는 완전히 구분되어 있다.
  - Read Data가 모두 도착하지 않은 상황에서 새로운 Request를 보낼 수 있다.
  - Write Data가 모두 전송되지 않은 상황에서 새로운 Request를 보낼 수 있다.
- Constraints
  - Fixed Data Bus Width(32bit or 64bit)
    - 모든 Request는 Data Bus width 단위로 일어난다.
    - 즉, byte/half-word transfer 등은 지원하지 않는다.
    - Write의 경우 Byte Lane별로 write할 수 있는 signal을 제공한다.
  - Only Support Incremental Burst
    - FIXED와 WRAP Burst는 지원하지 않는다.
  - Configurable Read-Only/Write-Only/Read-Write
  - No Support for AXI ID
    - AXI ID는 지원하지 않는다.
  - No Support for Write Response
    - Write Response를 check하지 않는다.
    - 현재의 BUS 구조에서는 Write Data가 모두 전송되면 Write가 끝나게 된다.
  - No Support for BUS Error Detection
    - BUS 에러를 detect하지 않는다. 일반적인 AXI Master는 BUS Error를 detect할 필요가 없다.

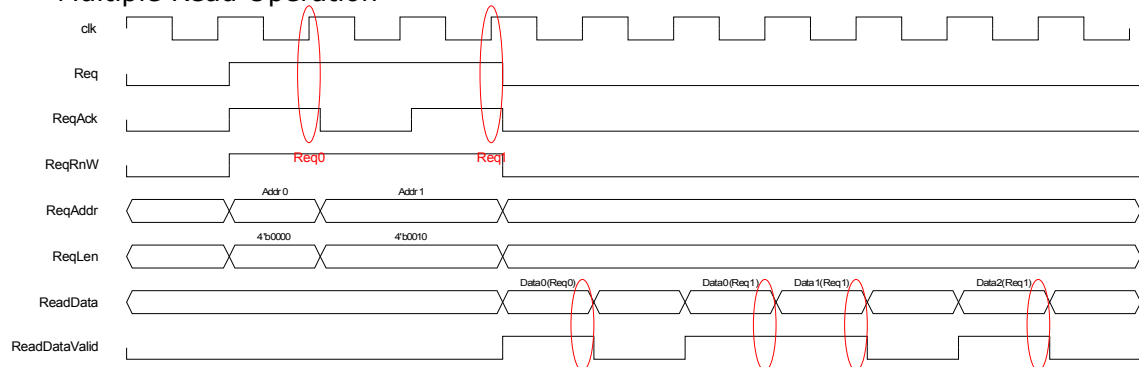
## (\*) Operation

### – Read Operation



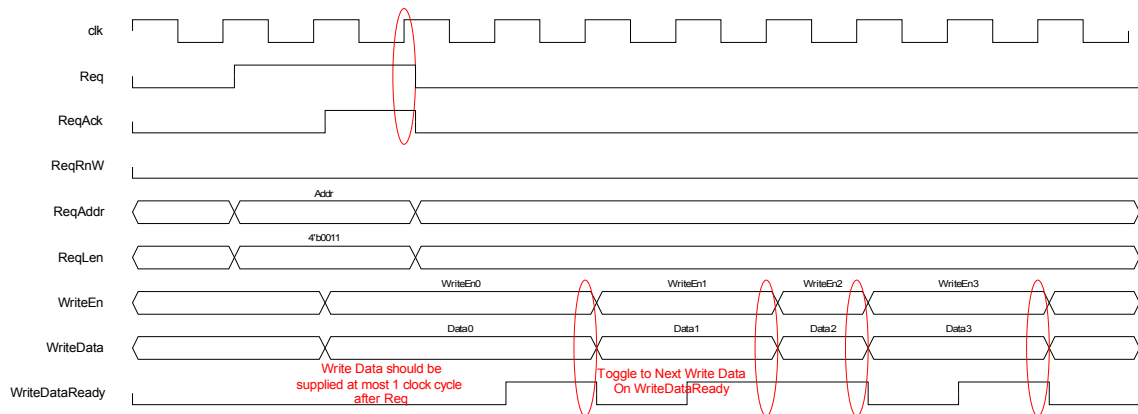
Read Operation은 위의 timing diagram처럼 동작하게 된다. 기본적으로 ReadAddr/ReqLen/ReqRnW(read의 경우 high)를 drive하고 Req를 assert하게 되면 AXI Bus Interface block에서 ReqAck를 assert하게 된다. Req signal과 ReqAck signal이 동시에 assert되었을 때 Request의 handshake가 일어나게 된다. Bus Interface Block에서 data를 전달할 때는 Data를 latch할 수 있도록 ReadDataValid라고 하는 signal과 같이 전달한다. ReadDataValid가 high일 때 ReadData를 latch하면 된다.

### – Multiple Read Operation



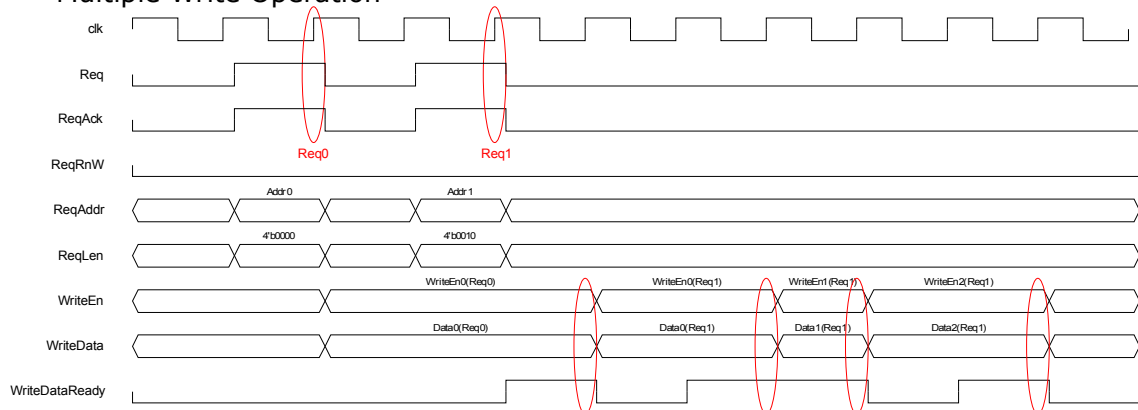
Multiple Read Operation은 위와 같은 timing diagram처럼 진행된다. 위의 timing diagram은 첫번째 request는 single request이고 두 번째 request는 burst 3 request이다. request와 read data가 전달되는 부분은 완전히 독립적으로 동작하므로 첫번째 request의 data가 도착하지 않은 상황에서도 두 번째 request를 보낼 수 있다. 두번째 Request는 Bus Interface 쪽에서 Ack를 1 cycle 뒤에 보내주고 있다. 실제로 몇 cycle뒤에 보내주게 될지는 BUS 상황에 따라서 다르므로 얼마나 latency가 생길지는 알 수 없다. request를 전달할 때 주의할 점은 Request가 assert되면 ReqAck가 asset될 때까지, ReqAddr/ReqRnW/ReqLen은 stable하게 유지되어야 한다는 점이다. ReadDataValid signal을 보고 ReadData를 latch하면 된다.

### – Write Operation



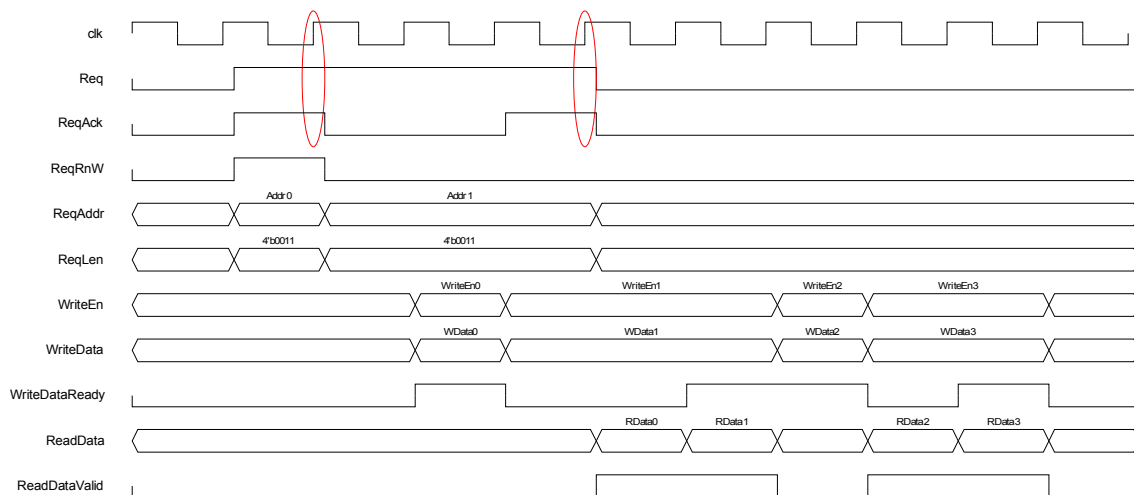
Write Operation은 위와 같은 timing diagram대로 진행된다. Request의 전달은 Read와 동일하고 WriteEn/WriteData를 BUS Interface쪽에 공급하면 된다. 두 신호의 공급은 req가 assert되는 시점과 동일하거나 적어도 1 cycle 안에 이루어져야 한다. BUS Interface쪽에서는 WriteDataReady라는 signal이 있어 공급된 WriteEn/WriteData signal을 전달했음을 알려주게 된다. 따라서 WriteDataReady가 assert되면 다음에 쓸 WriteData와 WriteEn으로 toggle해 주면 된다.

#### – Multiple Write Operation



Multiple Write Operation의 동작은 위와 같은 timing diagram과 같다.

#### – Read&Write Operation



위의 timing diagram처럼 Read Operation과 Write Operation이 섞여서 진행될 수도 있다. Read Data가 전달되는 부분과 Write Data를 전달하는 부분은 완전히 다르므로 Request 순서와 관계 없이 진행됨을 유의해야한다. 위의 timing diagram에서 또 독특한 것은 Write Request(두 번째 request)의 handshake가 일어나기 전에 WriteDataReady가 assert될 수도 있다는 점이다. 따라서 WriteData의 공급과 toggle은 ReqAck가 assert될 때까지 기다려서 수행하면 안된다. 마찬가지로 Read Operation에서도 Request Handshake가 이루어지기 전에 Read Data가 전달될 수 있으므로 주의해야 한다.

#### (\*) Configuration

우선 verilog file(AXIBUSIf.v) 하나로 되어있고, AXIBUSIf라는 단일 module로 구성된다. 이 module의 동작을 Configuration할 수 있는 macro가 있는데 verilog file 맨 처음에 있고 다음과 같다.

| <b>macro</b>   | <b>Allowed Value</b> | <b>Description</b>                                            |
|----------------|----------------------|---------------------------------------------------------------|
| AXIBUSIF_READ  | any                  | 이 macro가 define되면 BUS Interface가 Read Request 처리를 할 수 있게 된다.  |
| AXIBUSIF_WRITE | any                  | 이 macro가 define되면 BUS Interface가 Write Request 처리를 할 수 있게 된다. |

기본적으로 두 macro중에 하나는 define되어야 한다. 당연히 두 macro 모두 define할 수 있는데, 이 경우에는 Read와 Write Request를 모두 처리할 수 있다. 모두 처리를 하더라도 AXI BUS쪽의 Write Address Channel과 Read Address Channel이 동시에 동작하는 것은 아니다. 즉, Request가 순차적으로 AXI Slave쪽으로 전달된다. 당연히, Read Data와 Write Data는 동시에 처리될 수 있다. 두 macro가 모두 define되면 ReqRnW라는 input signal이 생기고 그것으로 현재 request가 read인지 write인지 선택할 수 있다. 두 macro중에 하나만

define되는 경우에는 ReqRnW input signal이 필요 없고 module interface에서도 빠지게 된다. AXIBUSIF\_READ만 define된 경우 모든 request를 read로 인식하고, AXIBUSIF\_WRITE만 define된 경우 모든 request를 write로 인식하게 된다. 만약 Read/Write request가 동시에 전달되는 구조를 원한다면, Read-Only로 configuration된 Bus Interface와 Write-Only로 Configuration된 Bus Interface 두 개를 사용하면 된다.

Configuration Parameter도 존재하는데 verilog module interface가 선언된 다음에 존재한다.

| <i><b>parameter</b></i> | <i><b>Allowed Value</b></i> | <i><b>Description</b></i>                                                                                                                                                                                                                                                                                                               |
|-------------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DATA_WIDTH              | 32 or 64                    | BUS Width를 Configuration하는 parameter로 32bit혹은 64bit으로 정해야 한다.                                                                                                                                                                                                                                                                           |
| FIFO_DEPTH              | Integer(>= 1)               | FIFO의 depth를 결정하는 값이다. $2^{(FIFO\_DEPTH)}$ 만큼의 fifo depth를 가지게 된다. 적어도 1이상의 값을 가져야 한다. FIFO는 AXI에서 사용되는 WLAST 신호 생성을 위해서 AWLEN를 저장하기 위해서 사용된다. 동시에 issue되는 Write Request의 갯수를 추산한 후에 그 값에 1을 더한 fifo depth의 크기를 결정한다. 그 fifo depth에 해당하도록 parameter의 값을 주도록 한다. 물론 성능에 관계가 없으면 1로 놔두어도 무방하다. 당연히 AXIBUSIF_WRITE가 define되어 있지 않으면 무의미하다. |

#### (\*) Signals

##### - Request Related

| <i><b>signal</b></i> | <i><b>방향</b></i> | <i><b>Description</b></i>                                                               |
|----------------------|------------------|-----------------------------------------------------------------------------------------|
| Req                  | IN               | Request Strobe(Active High)<br>Request가 있을 때 Req 신호를 high로 만들어 준다.                      |
| ReqAddr[31:0]        | IN               | Request Address<br>DATA_WIDTH가 32일때는 하위 2bit, DATA_WIDTH가 64일때는 하위 3 bit가 무시된다.         |
| ReqLen[3:0]          | IN               | Request Burst Length<br>Incremental Burst Length로 0일때 Single, 15일때 16 beat burst가 수행된다. |
| ReqRnW               | IN               | Request Type(1이면 Read, 0이면 Write)                                                       |

| <b>signal</b>                      | <b>방향</b> | <b>Description</b>                                                                                                                                                                                                                 |
|------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                    |           | Optional Signal로 AXIBUSIF_READ, AXIBUSIF_WRITE가 동시에 define된 경우 존재하고 그렇지 않으면 존재하지 않는다.                                                                                                                                              |
| ReqAck                             | OUT       | Request Acknowledge(Active High)<br>ACLK의 rising edge에서 Req와 ReqAck가 모두 High일 때 Request의 handshake가 일어나고 사용자는 새로운 Request를 Req를 assert하여 보낼 수 있다. Req가 high이고 ReqAck가 low로 유지되는 때에는 Req/ReqAddr/ReqLen/ReqRnW는 stable하게 유지 되어야 한다. |
| WriteData[31:0]<br>WriteData[63:0] | IN        | Write Data(DATA_WIDTH만큼의 폭을 가진다)<br>사용자가 공급해야 하는 WriteData로 WriteDataReady가 high가 되면 다음 WriteData로 toggle되어야 한다.                                                                                                                   |
| WriteEn[3:0]<br>WriteEn[7:0]       | IN        | Write Byte Enable(DATA_WIDTH/8 만큼의 폭을 가진다)<br>Active High이고 WriteDataReady가 high가 되면 다음 WriteEn으로 toggle되어야 한다.                                                                                                                    |
| WriteDataReady                     | OUT       | Write Data Ready<br>ACLK의 rising edge에서 WriteDataReady가 high이면 WriteData와 WriteEn을 다음에 쓸 data로 toggle해야 한다.                                                                                                                        |
| ReadData[31:0]<br>ReadData[63:0]   | OUT       | Read Data(DATA_WIDTH만큼의 폭을 가진다)<br>AXI Interface로 읽어온 Data로 ReadDataValid가 high일 때 ACLK의 rising edge에서 latch하면 된다.                                                                                                                 |
| ReadDataValid                      | OUT       | Read Data Valid(Active High)<br>ReadData가 valid하다는 것을 나타내는 flag이고 ACLK의 rising edge에서 유효하다.                                                                                                                                        |

#### – AXI Master Interface

AXI Master Interface에 대한 설명은 생략한다.

#### (\*) Integration시 주의 사항

##### – Request Signal

Req가 일단 assert되면 Request와 관련된 signal(Req, ReqLen, ReqAddr, ReqRnW)는 ReqAck가 assert될 때까지 stable하게 유지되어야 한다.

##### – Req & ReqAck

idle 상태에서는 ReqAck가 1이 될때까지 기다렸다가 Req를 assert하면 안된다. 물론 다음 request는 Req와 ReqAck가 동시에 assert된 후에 생성해야 하지만, 현재 Req가 1이 아닌데도 ReqAck가 1이 될 때까지 기다렸다가 Req를 1로 assert하면 안된다. 현재 ReqAck는 Req가 1일 때만 assert되도록 설계되어 있으므로 ReqAck가 1로 assert될 때까지 기다리면 infinite loop에 빠지게 된다.

– Read Data or Write Data before ReqAck assertion

BUS Interface의 내부 처리로 인해 Request가 handshake(ACLK의 rising edge에서 Req와 ReqAck이 모두 1인 경우)가 되기 전에 Write Data를 요구(WriteDataReady가 high)되거나 Read Data가 도착(ReadDataValid가 high)할 수 있으므로 주의해야 한다. 될 수 있으면 Write Data의 준비는 Req를 assert하는 시점과 동일하게 맞추고, Read Data를 사용자의 FIFO에 쓸 준비를 Req가 assert하는 시점과 동일하게 맞추는 것이 좋다. 단, 1 clock(ACLK) cycle의 차이는 있어도 된다. 즉, Req가 assert된 cycle 다음 cycle부터 Write Data가 VALID 해도 된다.

– module naming

AXIBUSIf라는 module이 전체 chip에서 여러 개 존재하면 곤란하므로 module이름을 각 AXI Master별로 다른 이름을 사용해야 한다. 추후의 Revision에 대해서 될 수 있으면 Configuration과 관련되지 않은 소스 코드는 수정하지 않기를 권고한다. 즉, 위에서 언급한 Configuration관련 macro와 parameter를 변경하고, AXIBUSIf라는 module이름에 prefix 또는 suffix를 붙여서(예를 들어 BitBLTWriteAXIBUSIf) 사용해야 한다.

– Timing

AXIBUSIf는 기본적으로 mealy machine으로 설계되어 있다. 따라서 input에 output까지 combinational하게 연결되는 경우가 생기게 된다. 따라서 합성결과 원하는 clock frequency에서 동작하지 않는다면 Request Line에 AXI Register Slice(Fully Registered)를 삽입하여 Request Line과 AXI Master Interface사이의 timing을 isolation시켜야 한다. AXI Register Slice는 기본적으로 VALID와 READY, INFORMATION이라는 신호로 interface 되는데, VALID를 Req, READY를 ReqAck, INFORMATION에 나머지 Request Signal(ReqLen, ReqAddr 등)을 연결하면 된다. Register Slice의 Integration에 관련한 자세한 내용은 Register Slice Integration Guide를 참조하라.